are sent to a unique location. In both cases, the messages must be immutable for the systems to work. This is because if a message can be changed before it is consumed, it distorts its intent, just as would happen if someone changed your vote in an election. In the case of events, changing them would make the history unreliable, as they represent facts.

Event- or message-driven systems benefit greatly from asynchrony and flow control in their components. This way, you can make sure the system throttles the producer if it outputs messages faster than the consumer processes them. You can find this in libraries like Akka Streams, Pivotal Reactor, and Vert.x that create reactive components. It is important to know that these components must all be chained together for the system to have reactive properties. If a component in the middle of the chain does not have flow control, messages will be lost, some components will halt, or both.

B.2.2 Architectural patterns

Event-driven architectures that treat transitions as a source of truth are called *event sourcing* (ES). This approach has many advantages, such as performance. Most databases are optimized for append-only. Another advantage is traceability: because all events are stored, you can prove—by replying to them—that your system behaved as prescribed, which is a legal requirement in many organizations. Another major advantage over in-place replacement is the flexibility to have multiple representations of the same set of events.

Imagine you have a data structure that represents the status of a shopping cart, and you write in it the number of items, their ID and descriptions, and the price for each change in the cart. If you use a